

14 - Dictionaries

Diccionarios

Los objetos de tipo `dict` (abreviatura de `dictionary`) son como listas excepto que los indexamos usando `keys` en lugar de enteros. Cualquier objeto `hashable` puede ser usado como una clave. Piensa en un diccionario como un conjunto de pares `clave/valor`. Los literales de tipo `dict` se encierran entre llaves y cada elemento se escribe como una `key` seguida de dos puntos seguido de un `value`.

📋 ¿Qué es un objeto hashable?

Un objeto `hashable` (en inglés, `hashable`) es aquel que:

1. Tiene un valor de hash que nunca cambia durante su vida (es decir, es `immutable`).
2. Puede ser comparado con otros objetos (es decir, implementa `__eq__`).
3. Puede usarse como `clave en un diccionario` (`dict`) o `elemento en un conjunto` (`set`).

¿Y qué es un hash?

Un `hash` es un número que representa de forma única a un objeto. Python lo obtiene usando la función incorporada `hash()`.

Si el objeto cambia (como una lista), su valor de hash también podría cambiar, lo cual rompe las reglas del diccionario, porque este necesita encontrar las claves rápidamente usando el hash.

✅ Son hashables: `int`, `float`, `str`, `bool`, `tuple` (si todos sus elementos también lo son), `frozenset`.

❌ No son hashables: `list`, `dict`, `set`, porque `pueden cambiar`.

Por ejemplo, el código:

```
1 month_numbers = {'Jan':1, 'Feb':2, 'Mar':3, 'Apr':4, 'May':5, 1:'Jan',
2                 2:'Feb', 3:'Mar', 4:'Apr', 5:'May'}
3 print(month_numbers)
4
```

```
5 print('The third month is ' + month_numbers[3])
6
7 dist = month_numbers['Apr'] - month_numbers['Jan']
8
9 print('Apr and Jan are', dist, 'months apart')
```

imprimirá:

```
1 {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 1: 'Jan', 2: 'Feb', 3:
  'Mar', 4: 'Apr', 5: 'May'}
2
3 The third month is Mar
4
5 Apr and Jan are 3 months apart
```

Acceder a un valor es tan simple como `d[clave]`. No se necesita recorrer la estructura. Las entradas en un `dict` no pueden ser accedidas usando un índice. Es por eso que `month_numbers[3]` se refiere inequívocamente a la entrada con la clave `3` en lugar de la cuarta entrada. Si una clave está definida en un diccionario puede ser probado usando el operador `in`.

❗ ¿Porque usar diccionarios?

Usar múltiples listas (nombres, notas, tareas) puede ser propenso a errores. Los diccionarios permiten almacenar esta información de forma más clara y directa: `{'Ana': {'nota': 'A', 'tareas': [8, 9]}}`.

Al igual que las listas, **los diccionarios son mutables**. Podemos agregar una entrada escribiendo, por ejemplo, `month_numbers['June'] = 6` o cambiar una entrada escribiendo, por ejemplo, `month_numbers['May'] = 'V'`.

Los diccionarios son una de las grandes cosas sobre Python. Reducen enormemente la dificultad de escribir una variedad de programas. Por ejemplo, en el código "Traduciendo texto (mal hecho)" usamos diccionarios para escribir un programa (bastante horrible) para traducir entre idiomas.

⚠ Claves inexistentes

Si la clave no existe, `d[clave]` lanza un `KeyError`. Usar `d.get(clave, valor_por_defecto)` es una alternativa segura.

El código en la figura imprime:

```
1 Je bois "good" rouge vin, et mange pain.
2 I drink of wine red.
```

Recuerda que los diccionarios son mutables. Por lo tanto, ten cuidado con los efectos secundarios. Por ejemplo,

```
1 FtoE['bois'] = 'wood'
2 print(translate('Je bois du vin rouge.', dicts, 'French to English'))
```

imprimirá:

```
1 I wood of wine red.
```

Traduciendo texto (mal hecho)

```
1 EtoF = {'bread':'pain', 'wine':'vin', 'with':'avec', 'I':'Je',
2        'eat':'mange', 'drink':'bois', 'John':'Jean', 'friends':'amis', 'and':
3        'et', 'of':'du', 'red':'rouge'}
4
5 FtoE = {'pain':'bread', 'vin':'wine', 'avec':'with', 'Je':'I',
6        'mange':'eat', 'bois':'drink', 'Jean':'John', 'amis':'friends',
7        'et':'and', 'du':'of', 'rouge':'red'}
8
9 dicts = {'English to French':EtoF, 'French to English':FtoE}
10
11 def translate_word(word, dictionary):
12     if word in dictionary:
13         return dictionary[word]
14     elif word != '':
15         return ' ' + word + ' '
16     return word
17
18 def translate(phrase, dicts, direction):
19     UC_letters = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
20     LC_letters = 'abcdefghijklmnopqrstuvwxyz'
21     punctuation = '.,;!?'
22     letters = UC_letters + LC_letters
23     dictionary = dicts[direction]
24     translation = ''
25     word = ''
26     for c in phrase:
27         if c in letters:
28             word = word + c
29         elif word != '':
30             if c in punctuation:
31                 c = c + ' '
32             translation = (translation + translate_word(word, dictionary)
33 + c)
34             word = ''
35     return f'{translation} {translate_word(word, dictionary)}'
36
37 print(translate('I drink good red wine, and eat bread.', dicts, 'English to
38 French'))
```

```
31 print(translate('Je bois du vin rouge.', dicts, 'French to English'))
```

Muchos lenguajes de programación no contienen un tipo incorporado que proporcione un mapeo de claves a valores. En su lugar, los programadores usan otros tipos para proporcionar funcionalidad similar. Es, por ejemplo, relativamente fácil implementar un diccionario usando una lista en la cual cada elemento es una tupla que representa un par clave/valor. Entonces podemos escribir una función simple que haga la búsqueda asociativa, por ejemplo,

```
1 def key_search(L, k):
2     for elem in L:
3         if elem[0] == k:
4             return elem[1]
5     return None
```

El problema con tal implementación es que es *computacionalmente ineficiente*. En el peor caso, un programa podría tener que examinar cada elemento en la lista para realizar una sola búsqueda. En contraste, la implementación incorporada es rápida. Usa una técnica llamada *hashing* para hacer la búsqueda en tiempo que es casi independiente del tamaño del diccionario.

Hay múltiples formas de usar una declaración `for` para iterar sobre las entradas en un diccionario. Si `d` es un diccionario, un bucle de la forma `for k in d` itera sobre las claves de `d`. El orden en el cual las claves son elegidas es el orden en el cual las claves fueron insertadas en el diccionario. Por ejemplo,

```
1 capitals = {'France': 'Paris', 'Italy': 'Rome', 'Japan': 'Kyoto'}
2 for key in capitals:
3     print('The capital of', key, 'is', capitals[key])
```

imprime:

```
1 The capital of France is Paris
2 The capital of Italy is Rome
3 The capital of Japan is Kyoto
```

Para iterar sobre los valores en un diccionario, podemos usar el método `values`. Por ejemplo,

```
1 cities = []
2 for val in capitals.values():
3     cities.append(val)
4 print(cities, 'is a list of capital cities')
```

```
imprime ['Paris', 'Rome', 'Kyoto'] is a list of capital cities .
```

El método `values` devuelve un objeto de tipo `dict_values`. Este es un ejemplo de un **view object** (objeto vista). Un objeto vista es dinámico en el sentido de que si el objeto con el cual está asociado cambia, el cambio es visible a través del objeto vista. Por ejemplo,

```
1 cap_vals = capitals.values()
2 print(cap_vals)
3 capitals['Japan'] = 'Tokyo'
4 print(cap_vals)
```

imprime:

```
1 dict_values(['Paris', 'Rome', 'Kyoto'])
2 dict_values(['Paris', 'Rome', 'Tokyo'])
```

De manera similar, el método `keys` devuelve un objeto vista de tipo `dict_keys`. Los objetos vista pueden ser convertidos a listas, por ejemplo, `list(capitals.values())` devuelve una lista de los valores en `capitals`.

Para iterar sobre pares clave/valor, usamos el método `items`. Este método devuelve un objeto vista de tipo `dict_items`. Cada elemento de un objeto de tipo `dict_items` es una `tuple` de una clave y su valor asociado. Por ejemplo, el código

```
1 for key, val in capitals.items():
2     print(val, 'is the capital of', key)
```

imprime:

```
1 Paris is the capital of France
2 Rome is the capital of Italy
3 Tokyo is the capital of Japan
```

Ejercicio manual

Implementa una función que cumpla con la especificación

```
1 def get_min(d):
2     """d es un dict que mapea letras a ints
3     devuelve el valor en d con la clave que ocurre primero
4     en el alfabeto. Por ejemplo, si d = {x = 11, b = 12}, get_min
5     devuelve 12."""
6     min_key = min(d)
```

A menudo es conveniente usar tuplas como claves. Imagina, por ejemplo, usar una tupla de la forma `(flight_number, day)` para representar vuelos de aerolíneas. Si quisiéramos entonces que fuera fácil usar tal tupla como clave en un diccionario implementando un mapeo de vuelos a horas de llegada. Una lista no puede ser usada como clave, porque los objetos de tipo `list` no son hasheables.

Algunas operaciones comunes en `dicts`

Operación	Descripción
<code>len(d)</code>	devuelve el número de elementos en d
<code>d.keys()</code>	devuelve una vista de las claves en d
<code>d.values()</code>	devuelve una vista de los valores en d
<code>d.items()</code>	devuelve una vista de los pares (clave, valor) en d
<code>d.update(d1)</code>	actualiza d con los pares (clave, valor) en d1, sobrescribiendo las claves existentes
<code>k in d</code>	devuelve True si la clave k está en d
<code>d[k]</code>	devuelve el elemento en d con la clave k
<code>d.get(k, v)</code>	devuelve d[k] si k está en d, y v en caso contrario
<code>d[k] = v</code>	asocia el valor v con la clave k en d. Si ya hay un valor asociado con k, ese valor es reemplazado
<code>del d[k]</code>	elimina la clave k de d
<code>d.copy()</code>	copiar un diccionario

Diccionarios anidados

Un **diccionario anidado** es un diccionario que tiene como valor... ¡otro diccionario! Es decir, una estructura jerárquica donde cada clave principal puede mapear a un conjunto de datos más complejo. Esta técnica es muy útil para modelar entidades que tienen múltiples atributos agrupados.

¿Cuándo se usan?

Cuando necesitas agrupar datos relacionados de forma ordenada. Por ejemplo:

- Información de estudiantes: nombre → {exámenes, trabajos, nota final}
- Productos en una tienda: producto → {precio, stock, categoría}
- Calendarios: día → {eventos, clima, feriados}

Para esto veamos un ejemplo:

```
1  grades = {  
2      'Ana':  {'mq': [5, 4, 4], 'ps': [10, 9, 9], 'final': 'B'},  
3      'Bob':  {'mq': [6, 7, 8], 'ps': [8, 9, 10], 'final': 'A'},  
4      'Eric': {'mq': [3],      'ps': [0],      'final': 'C'}  
5  }
```

- La clave de primer nivel es el **nombre del estudiante**
- El valor asociado es otro **diccionario** que agrupa distintos tipos de notas (**mq**, **ps**, **final**)

Para acceder a un valor dentro de esta estructura, simplemente vas bajando los niveles:

```
1  grades['Ana']           # {'mq': [5, 4, 4], 'ps': [10, 9, 9],  
   'final': 'B'}  
2  grades['Ana']['mq']     # [5, 4, 4]  
3  grades['Ana']['mq'][0]  # 5  
4  grades['Bob']['final']  # 'A'
```

Si quisiéramos, por ejemplo, imprimir todas las notas finales:

```
1  for student, info in grades.items():  
2      print(student, '→ Nota final:', info['final'])
```

Que imprimiría

```
1  Ana → Nota final: B  
2  Bob → Nota final: A
```

⚠ ¿Y si anido más niveles?

¡Es válido! Python permite anidar estructuras tan profundas como necesites. Pero si te encontrás accediendo a algo como `data[a][b][c][d]`, tal vez deberías considerar simplificar.

Ejercicio manual de la lectura

Implemente la función que cumpla las siguientes especificaciones:

```

1  def keys_with_value(aDict, target):
2      """
3      aDict: a dictionary
4      target: an integer or string
5      Assume that keys and values in aDict are integers or strings.
6      Returns a sorted list of the keys in aDict with the value target.
7      If aDict does not contain the value target, returns an empty list.
8      """
9      result = []
10     for key in aDict:
11         if aDict[key] == target:
12             result.append(key)
13     return sorted(result)
14
15 # Examples:
16 aDict = {1:2, 2:4, 5:2}
17 target = 2
18 print(keys_with_value(aDict, target)) # prints the list [1,5]
```

Implemente la función que cumpla las siguientes especificaciones:

```

1  def all_positive(d):
2      """
3      d is a dictionary that maps int:list
4      Suppose an element in d is a key k mapping to value v (a non-empty
5      list).
6      Returns the sorted list of all k whose v elements sums up to a
7      positive value.
8      """
9      result = []
10
11     for key in d:
12         if sum(d[key]) > 0:
13             result.append(key)
14
15     return sorted(result)
```



```
15
16 # Examples:
17 d = {5:[2,-4], 2:[1,2,3], 1:[2]}
18 print(all_positive(d)) # prints the list [1, 2]
```